

第十九届全国大学生信息安全竞赛（作品赛）
暨第三届“长城杯”网数智安全大赛（作品赛）
作品报告

☐ 命题赛道

☒ 自由赛道

Food5作品名称：面向外卖平台的可控匿名订单认证与可信审计通信系统

电子邮箱：3863102544@qq.com

提交日期：2026 年 7 月 3 日

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目 录

摘要	4
第一章 作品概述	5
1.1 背景分析	5
1.2 相关工作与现有方案不足	5
1.3 作品目标	5
1.4 我们的工作	5
1.5 应用前景	5
第二章 预备知识	6
2.1 HMAC 消息认证码	6
2.2 SM3 密码杂凑算法	7
2.3 SM4 对称加密算法	8
2.4 PID 匿名身份机制	8
2.5 WebSocket 实时通信机制	9
2.6 Merkle Tree 完整性验证	10
2.7 条件溯源与可控匿名模型	11
第三章 系统设计	13
3.1 系统总体方案	13
3.2 系统参与方与信任边界	13
3.3 威胁模型	13
3.4 安全目标	13
3.5 系统总体架构	13
3.6 数据库设计	13
3.7 用户注册与防机器注册机制	13
3.8 PID 匿名身份生成机制	13
3.9 订单 Token 认证机制	13
3.10 用户端与骑手端匿名通信机制	13
3.11 消息加密存储与哈希摘要机制	13
3.12 Merkle 日志完整性验证机制	13
3.13 条件溯源与管理审计机制	13
3.14 三端摘要一致性增强机制	13

3.15	方案安全性分析	13
第四章	作品实现	14
4.1	系统开发环境	14
4.2	后端服务实现	14
4.3	用户端功能实现	14
4.4	骑手端功能实现	14
4.5	管理员端功能实现	14
4.6	注册、下单与订单认证流程实现	14
4.7	WebSocket 通信流程实现	14
4.8	Merkle 快照与完整性验证实现	14
4.9	条件溯源流程实现	14
4.10	系统运行界面	14
第五章	作品测试与分析	15
5.1	测试环境	15
5.2	功能测试	15
5.3	安全性测试	15
5.4	日志篡改检测测试	15
5.5	越权访问测试	15
5.6	性能测试	15
5.7	对比分析	15
第六章	功能展示与创新性说明	16
6.1	场景创新	16
6.2	机制创新	16
6.3	安全模型创新	16
6.4	系统闭环创新	16
6.5	应用价值	16
第七章	总结	17
7.1	作品总结	17
7.2	未来展望	17
参考文献		18

摘要

(请简要说明创作本作品之动机、功能、特性、创新处、实用性)

关键词：外卖平台；可控匿名；订单认证；PID；HMAC-SM3；Merkle Tree；条件溯源；可信审计

作品概述

1.1 背景分析

1.2 相关工作与现有方案不足

1.3 作品目标

1.4 我们的工作

1.5 应用前景

预备知识

本章围绕 FoodShield 系统设计中所涉及的核心技术进行介绍, 主要包括 HMAC 消息认证码、SM3 密码杂凑算法、SM4 对称加密算法、PID 匿名身份机制、WebSocket 实时通信机制、Merkle Tree 完整性验证机制以及条件溯源与可控匿名模型。上述技术共同支撑了本作品在外卖订单通信场景下的身份匿名、订单认证、消息保护、日志审计和异常追责等安全目标。

2.1 HMAC 消息认证码

HMAC (Hash-based Message Authentication Code) 是一种基于密码杂凑函数和共享密钥构造的消息认证码机制。与单纯的哈希运算不同, HMAC 在计算过程中引入了密钥, 因此不仅能够检测消息内容是否被篡改, 还能够验证消息是否由持有合法密钥的一方生成。HMAC 通常用于身份认证、接口鉴权、请求防伪造和数据完整性校验等场景。

设 H 表示密码杂凑函数, K 表示共享密钥, m 表示待认证消息, 则 HMAC 的一般形式可以表示为:

$$\text{HMAC}(K, m) = H((K' \oplus opad) \parallel H((K' \oplus ipad) \parallel m)) \quad (1)$$

其中, K' 表示经过填充或压缩处理后的密钥, $opad$ 和 $ipad$ 分别表示外部填充值和内部填充值, $\parallel$ 表示字符串连接操作。由于攻击者在不知道密钥 K 的情况下难以构造合法的认证码, 因此 HMAC 可以有效防止伪造认证凭证。

在 FoodShield 系统中, HMAC 主要用于订单 Token 的生成与验证。用户在创建订单后, 平台服务器根据订单号、匿名身份标识、时间戳和随机数等字段生成订单认证 Token。用户进入订单通信通道时需要提交订单号和 Token, 服务器重新计算并比对 Token, 以判断用户是否为该订单的合法参与方。该机制使用户能够在不直接暴露真实身份的情况下证明自己拥有订单通信资格。

本系统中订单 Token 的抽象形式如下:

$$m_{order} = orderID \parallel PID \parallel timestamp \parallel nonce \quad (2)$$

$$token = \text{HMAC}(K_{order}, m_{order}) \quad (3)$$

其中, K_{order} 为订单认证密钥, $orderID$ 为订单编号, PID 为用户匿名身份标识, $timestamp$ 为时间戳, $nonce$ 为随机数。通过引入时间戳和随机数, 可以降低 Token 被重放利用的风险; 通过绑定订单号与匿名身份标识, 可以防止攻击者将一个订单的认证凭证迁移到其他订单中使用。

2.2 SM3 密码杂凑算法

SM3 是我国商用密码标准中的密码杂凑算法, 输出长度为 256 bit, 主要用于数据完整性校验、数字签名、消息认证码构造和随机数生成等场景。密码杂凑算法具有单向性、抗碰撞性和雪崩效应等重要性质。单向性是指给定哈希值难以反推出原始输入; 抗碰撞性是指难以找到两个不同输入使其具有相同哈希值; 雪崩效应是指输入发生微小变化时, 输出结果会发生显著变化。

设 m 表示输入消息, SM3 的输出可以表示为:

$$h = \text{SM3}(m) \quad (4)$$

其中, h 为长度为 256 bit 的消息摘要。由于摘要长度固定, SM3 适合用于对任意长度数据生成紧凑的完整性标识。

在 FoodShield 系统中, SM3 主要用于三个方面。第一, SM3 可作为 HMAC 的底层杂凑函数, 用于构造 HMAC-SM3 订单认证 Token。第二, 系统可以对通信消息、密文内容、订单编号、发送方角色、时间戳等字段计算消息摘要, 作为日志完整性验证的基础。第三, 系统可以对 PID、设备标识等敏感字段进行摘要化处理, 使管理员端在默认情况下只看到摘要值, 而不直接接触敏感原始信息。

例如, 对一条订单通信消息, 可以将其摘要定义为:

$$message_{hash} = \text{SM3}(orderID \parallel senderRole \parallel timestamp \parallel ciphertext) \quad (5)$$

其中, $senderRole$ 表示发送方角色, $ciphertext$ 表示加密后的消息内容。只要消息内容、订单号、发送方角色或时间戳发生变化, 重新计算得到的摘要值就会发生变化。因此, 消息摘要可以作为后续 Merkle Tree 日志完整性验证的基本单元。

2.3 SM4 对称加密算法

SM4 是我国商用密码标准中的分组对称加密算法，分组长度为 128 bit，密钥长度为 128 bit。对称加密算法的特点是加密和解密使用相同密钥，具有计算效率高、适合批量数据加密等优点，常用于通信数据保护、数据库字段加密和文件加密等场景。

设 K 表示对称加密密钥， M 表示明文消息， C 表示密文消息，则 SM4 加密与解密过程可以抽象表示为：

$$C = \text{Enc}_K(M) \quad (6)$$

$$M = \text{Dec}_K(C) \quad (7)$$

在实际系统中，SM4 通常需要结合工作模式使用，例如 CBC、CTR、GCM 等模式。以 CBC 模式为例，加密时需要引入初始化向量 IV ，使相同明文在不同加密过程中得到不同密文，从而降低明文模式泄露风险。其抽象形式如下：

$$C = \text{SM4-CBC-Enc}_K(IV, M) \quad (8)$$

在 FoodShield 系统中，SM4 主要用于订单通信消息的加密存储。用户与骑手之间的消息经过服务器处理后，可以使用订单级会话密钥或平台侧加密密钥进行加密，再写入数据库。管理员端默认不直接展示消息明文，而是展示消息哈希、时间戳、订单号和 Merkle Root 等审计信息。这样可以降低数据库泄露或管理员越权查看造成的隐私风险。

需要说明的是，本作品原型系统以可运行、可展示和安全机制闭环为主要目标。在实际部署环境中，通信链路还应结合 HTTPS/TLS 保障传输安全，SM4 更多用于服务端存储阶段的消息内容保护。通过“传输层加密 + 存储层加密 + 日志哈希审计”的组合设计，可以从多个层面降低订单通信数据泄露和篡改风险。

2.4 PID 匿名身份机制

PID (Pseudonymous Identifier) 即伪身份标识或匿名身份标识，是隐私保护系统中常用的一种身份解耦机制。其核心思想是：系统内部不直接使用真实用户身份参与通信和认证，而是为用户生成一个与真实身份存在受控映射关系的匿名标识。在

日常业务过程中，通信参与方只能看到订单相关信息或匿名身份信息，不能直接获得用户真实身份。

在 FoodShield 系统中，用户注册后由平台服务器生成 PID。PID 可以由主密钥、用户真实标识、随机数和设备摘要等字段通过 HMAC 或哈希机制生成，其抽象形式如下：

$$PID = \text{HMAC}(K_{\text{master}}, \text{userID} \parallel r) \quad (9)$$

其中， K_{master} 为平台主密钥， userID 为用户真实身份标识， r 为随机数。引入随机数可以避免相同用户标识在不同场景下生成完全相同的匿名标识，从而降低跨订单关联风险。

PID 的作用不是取代用户登录身份，而是在订单通信过程中隐藏真实身份。用户在系统中仍然需要通过用户名、口令和设备识别等方式完成正常登录；登录成功后，系统在内部使用 PID 与订单进行绑定。骑手端只需要知道订单编号、订单状态和必要配送信息，不应看到用户真实身份，也不应直接看到 PID。管理员端默认也不直接展示 PID，而是展示经过摘要化处理的 PID 哈希值。

因此，FoodShield 中 PID 的设计原则可以概括为三点。第一，PID 是系统内部匿名标识，不作为普通前端页面中的公开身份展示。第二，PID 与真实身份之间的映射关系只能由平台服务器在受控条件下查询。第三，当发生投诉、恶意骚扰、纠纷处理或审计需求时，系统可以通过条件溯源流程恢复必要身份信息，从而实现隐私保护与责任追踪之间的平衡。

2.5 WebSocket 实时通信机制

WebSocket 是一种基于 TCP 的全双工通信协议，适用于浏览器与服务器之间的实时数据交互。传统 HTTP 通信通常采用“客户端请求、服务器响应”的模式，服务器难以主动向客户端推送消息。WebSocket 在客户端与服务器建立连接后，可以在同一连接上进行双向通信，因此常用于在线聊天、实时通知、协同编辑和数据监控等场景。

WebSocket 通信的一般过程包括连接建立、身份认证、消息发送、消息转发和连接关闭。客户端首先向服务器发起 WebSocket 握手请求，握手成功后双方建立长连接。随后，客户端和服务器可以在该连接上持续交换消息，而不需要为每条消息重新建立 HTTP 请求。

在 FoodShield 系统中，WebSocket 用于实现用户端与骑手端之间的订单级实时

通信。系统以订单号为基本单位建立通信房间，每个订单对应一个独立的通信通道。用户或骑手进入通信房间前，服务器需要验证其身份状态和订单参与资格。对于用户端，服务器需要校验订单号和 Token；对于骑手端，服务器需要校验其是否为该订单绑定的配送方。只有验证通过的参与方才能加入对应订单房间。

订单通信过程可以抽象为：

$$Client \xrightarrow{orderID, credential} Server \quad (10)$$

$$Server \xrightarrow{verify(orderID, credential)} Room(orderID) \quad (11)$$

$$Message \xrightarrow{forward} User/Rider \quad (12)$$

其中，*credential* 表示订单认证凭证或骑手身份凭证。服务器在转发消息的同时，对消息进行加密存储、哈希计算和日志记录，使通信功能与安全审计机制形成联动。

WebSocket 本身主要解决实时通信问题，并不直接等价于安全通信。因此，在实际应用中，WebSocket 应结合 HTTPS/TLS 形成 WSS 安全连接，以防止通信内容在传输过程中被窃听或篡改。本作品在系统设计中将 WebSocket 作为实时消息通道，将 Token 认证、消息加密、哈希审计和权限控制作为安全增强机制，从而避免将普通聊天功能误认为完整安全机制。

2.6 Merkle Tree 完整性验证

Merkle Tree 是一种基于哈希函数构造的树形数据完整性验证结构，又称哈希树。其基本思想是：先对每个数据块计算哈希值，作为叶子节点；再将相邻节点哈希拼接后继续计算父节点哈希；最终逐层向上计算得到根节点哈希，即 Merkle Root。只要任意一个叶子节点对应的数据发生变化，最终计算得到的 Merkle Root 就会发生变化。因此，Merkle Tree 常用于区块链、文件完整性验证、日志审计和分布式存储等场景。

设系统中共有 n 条消息日志，分别为 $m_1, m_2, \dots, m_n$ ，对应叶子节点可以表示为：

$$leaf_i = H(m_i) \quad (13)$$

相邻两个叶子节点进一步计算父节点：

$$parent_{i,j} = H(leaf_i \parallel leaf_j) \quad (14)$$

最终得到根节点：

$$root = MerkleRoot(leaf_1, leaf_2, \dots, leaf_n) \quad (15)$$

在 FoodShield 系统中，Merkle Tree 用于订单通信日志的完整性验证。系统并不直接将明文消息作为叶子节点，而是将与消息相关的摘要信息作为叶子节点输入。为了绑定订单上下文、发送方角色、消息顺序和密文内容，本系统中单条消息日志的叶子节点可以抽象定义为：

$$leaf_i = SM3(orderID \parallel msgSeq \parallel senderRole \parallel timestamp \parallel ciphertextHash \parallel prevHash) \quad (16)$$

其中，*orderID* 表示订单编号，*msgSeq* 表示消息序号，*senderRole* 表示发送方角色，*timestamp* 表示发送时间，*ciphertextHash* 表示消息密文摘要，*prevHash* 表示前一条消息日志摘要。该设计能够同时检测消息内容篡改、消息顺序调整和消息删除等异常情况。

为了保证 Merkle Root 的可验证性，系统采用快照机制保存 Root。每当订单通信结束、管理员手动生成审计快照或消息数量达到设定阈值时，系统对当前订单的消息日志集合计算 Merkle Root，并将订单号、消息数量、Root 值和快照时间写入日志快照表。后续进行完整性验证时，系统重新读取订单消息日志并计算新的 Root，与历史快照中的 Root 进行比较。如果二者一致，说明当前日志集合与快照生成时保持一致；如果二者不一致，则说明日志可能被修改、删除、插入或重排。

因此，Merkle Tree 在本系统中的作用不是加密消息内容，而是提供日志完整性证明。它可以使平台在发生纠纷时证明通信记录是否被篡改，从而提升订单通信记录的可信度和审计价值。

2.7 条件溯源与可控匿名模型

匿名机制能够保护用户隐私，但完全匿名也可能导致恶意行为难以追责。在外卖订单通信场景中，用户和骑手之间可能发生恶意骚扰、虚假投诉、配送纠纷或违

规沟通等情况。如果系统只强调匿名而不保留治理能力，平台将难以处理异常事件；如果系统默认暴露真实身份，则又会削弱隐私保护效果。因此，本作品采用条件溯源与可控匿名相结合的设计思路。

可控匿名是指系统在正常业务流程中隐藏用户真实身份，而在满足特定条件时，由具备权限的管理方按照审计流程恢复必要身份信息。该模型强调“日常匿名、异常可追责”。在 FoodShield 系统中，普通通信状态下，骑手端无法看到用户真实身份，也无法看到用户 PID；管理员端默认只查看订单摘要、消息哈希、Merkle Root 和完整性验证结果。当发生投诉、纠纷或平台审计需求时，用户或骑手可以基于订单号提交溯源申请，管理员经过身份认证和权限校验后，才能触发条件溯源流程。

条件溯源流程可以抽象为：

$$orderID \rightarrow PID \rightarrow userID \quad (17)$$

其中， $orderID$ 是业务入口， PID 是系统内部匿名标识， $userID$ 是真实用户身份标识。系统不鼓励管理员直接通过 PID 发起溯源，而是以订单号作为溯源入口。这种方式更符合外卖平台的真实业务逻辑，也可以降低 PID 被滥用或过度暴露的风险。

在条件溯源过程中，系统应记录管理员的关键操作，包括登录行为、查看订单摘要、执行完整性验证、确认溯源申请和查询身份映射等。每一次溯源操作都应写入审计日志，记录操作者、操作时间、目标订单、溯源理由和操作结果。通过这种方式，系统不仅可以追踪用户或骑手的异常行为，也可以约束管理员权限，避免审计能力本身成为新的隐私风险。

综上，条件溯源与可控匿名模型是 FoodShield 系统区别于普通匿名通信系统的重要设计。它既避免了普通外卖聊天系统中过度暴露身份的问题，也避免了完全匿名系统中责任主体无法追踪的问题，从而在隐私保护、订单认证和平台治理之间形成平衡。

系统设计

- 3.1 系统总体方案
- 3.2 系统参与方与信任边界
- 3.3 威胁模型
- 3.4 安全目标
- 3.5 系统总体架构
- 3.6 数据库设计
- 3.7 用户注册与防机器注册机制
- 3.8 PID 匿名身份生成机制
- 3.9 订单 Token 认证机制
- 3.10 用户端与骑手端匿名通信机制
- 3.11 消息加密存储与哈希摘要机制
- 3.12 Merkle 日志完整性验证机制
- 3.13 条件溯源与管理员审计机制
- 3.14 三端摘要一致性增强机制
- 3.15 方案安全性分析

作品实现

- 4.1 系统开发环境
- 4.2 后端服务实现
- 4.3 用户端功能实现
- 4.4 骑手端功能实现
- 4.5 管理员端功能实现
- 4.6 注册、下单与订单认证流程实现
- 4.7 WebSocket 通信流程实现
- 4.8 Merkle 快照与完整性验证实实现
- 4.9 条件溯源流程实现
- 4.10 系统运行界面

作品测试与分析

(建议包括测试方案、测试环境搭建、测试设备、测试数据、结果分析等)

5.1 测试环境

5.2 功能测试

5.3 安全性测试

5.4 日志篡改检测测试

5.5 越权访问测试

5.6 性能测试

5.7 对比分析

功能展示与创新性说明

(本部分内容主要说明作品的创新性)

6.1 场景创新

6.2 机制创新

6.3 安全模型创新

6.4 系统闭环创新

6.5 应用价值

总结

7.1 作品总结

7.2 未来展望

参考文献